

微服務彩虹大亂鬥

來源：<https://codelabs.developers.google.com/codelabs/rainbowrumpus?hl=zh-tw>

步驟數：9

瞭解 Google Cloud：在 Cloud Run 中部署微服務並加入虛擬 rumpu，讓微服務在其他微服務中擲回「彩虹」，同時贏得勝利！您將會實際部署 Kotlin、Java、Go、Python 或 Node.js 微服務，並在過程中學習容器和 Cloud Run。持續改善演算法，看看您是否能比其他冒險者多獲得分數。

目錄

1. [1. 簡介](#)
2. [2. 登入 Google Cloud](#)
3. [3. 部署微服務](#)
4. [4. 要求將應用程式納入競技場](#)
5. [5. 進行及部署變更](#)
6. [6. 在本機開發 \(選用\)](#)
7. [7. 持續推送軟體更新](#)
8. [8. 觀測能力](#)
9. [9. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

上次更新時間：2021 年 5 月 6 日

微服務彩虹大亂鬥

你是否曾參與雪仗，在移動的同時，向其他人丟雪球？如果還沒用過，不妨找機會試試！但現在您不必擔心會被實體擊中，可以建構一個可透過網路存取的小型服務 (微服務)，與其他微服務展開史詩般的戰鬥，投擲彩虹而非雪球。

你可能想知道... 但微服務如何「向」其他微服務「拋出」彩虹？微服務可以接收網路要求 (通常是透過 HTTP)，並傳回回應。系統會提供「競技場管理員」，將競技場的目前狀態傳送給微服務，然後微服務會傳回指令，指定要執行的動作。

當然，目標是贏得勝利，但過程中您會瞭解如何在 Google Cloud 上建構及部署微服務。

運作方式

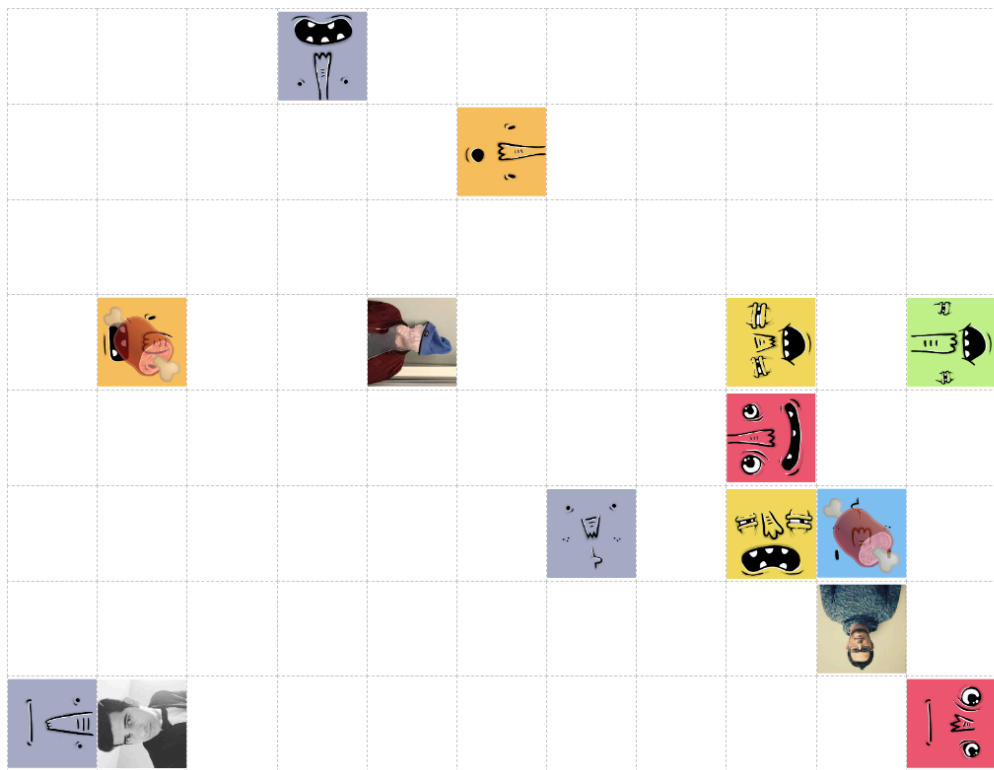
您可以使用任何技術建構微服務 (或從 Go、Java、Kotlin、Scala、NodeJS 或 Python 入門範本中選擇)，然後在 Google Cloud 上部署微服務。部署完成後，請將微服務的網址告知我們，我們會將其新增至競技場。

競技場包含特定戰鬥的所有玩家。彩虹大亂鬥將有自己的競技場。每位玩家代表一項微服務，會四處移動並向其他玩家投擲彩虹。

競技場管理員大約每秒會呼叫一次微服務，傳送目前的競技場狀態 (玩家所在位置)，而微服務會回覆要執行的指令。在競技場中，您可以前進、向左或向右轉，或是投擲彩虹。彩虹會朝玩家面向的方向移動，最多可移動三格。如果彩虹「擊中」其他玩家，投擲者可得一分，被擊中的玩家則會扣一分。系統會根據目前的玩家人數自動調整競技場大小。

過去的競技場如下所示：

Spring I/O - Battle Jamón



High Scores 🏆

	judomu	11ms	14
	Prashant Bhuru	39ms	13
	Daniel	16ms	11
	Anbu	11ms	2
	Stefano	11ms	1
	DomenP	119ms	0
	Antoni	14ms	-1
	Hatim	14ms	-2
	David	12ms	-4
	Geostyl	10ms	-5
	Sewerin	11ms	-6
	Simone	17ms	-6
	Denniss	11ms	-8
	Ladislav	37ms	-9

範例：第一場戰鬥的競技場

循環衝突

在競技場中，多位玩家可能會嘗試執行衝突的動作。舉例來說，兩位玩家可能會嘗試移動到同一個空間。如有衝突，回應時間最短的微服務會勝出。

觀看對戰

如要查看微服務在戰鬥中的表現，請[前往即時競技場](#)！

Battle API

如要與競技場管理員合作，微服務必須實作特定 API，才能參與競技場。競技場管理員會透過 HTTP POST 將目前的競技場狀態傳送至您提供的網址，JSON 結構如下：

```
{
  "_links": {
    "self": {
      "href": "https://YOUR_SERVICE_URL"
    }
  },
  "arena": {
    "dims": [4,3], // width, height
    "state": {
      "https://A_PLAYERS_URL": {
        "x": 0, // zero-based x position, where 0 = left
        "y": 0, // zero-based y position, where 0 = top
        "direction": "N", // N = North, W = West, S = South, E = East
        "wasHit": false,
        "score": 0
      }
      ... // also you and the other players
    }
  }
}
```

HTTP 回應必須是狀態碼 200 (OK)，且回應內文包含您的下一步行動，並編碼為下列其中一個大寫字元：

```
F <- move Forward
R <- turn Right
L <- turn Left
T <- Throw
```

就是這麼簡單！接下來，我們將逐步說明如何在 [Cloud Run](#) 上部署微服務。Cloud Run 是 Google Cloud 服務，可執行微服務和其他應用程式。

步驟 2 · 約 5 分鐘

2. 登入 Google Cloud

如要在 Cloud Run 上部署微服務，您必須登入 Google Cloud。我們會將抵免額套用至你的帳戶，你不需要輸入信用卡資訊。使用個人帳戶 (例如 gmail.com) 通常比使用 G Suite 帳戶更不容易發生問題，因為有時 G Suite 管理員會禁止使用者使用特定 Google Cloud 功能。此外，我們使用的網頁控制台應可順暢搭配 Chrome 或 Firefox 運作，但可能無法在 Safari 中正常運作。

3. 部署微服務

只要微服務可公開存取並符合 Battle API 規定，您就能使用任何技術建構微服務，並部署至任何位置。不過，為了簡化流程，我們會協助您從範例服務著手，並將其部署至 Cloud Run。

選擇要開始使用的範例

您可以從許多戰鬥微服務範例著手：

Kotlin 和 Spring Boot	資料來源	
Kotlin 和 Micronaut	資料來源	
Kotlin 和 Quarkus	資料來源	
Java 和 Spring Boot	資料來源	
Java 和 Quarkus	資料來源	
Go	資料來源	
Node.js 和 Express	資料來源	
Python 和 Flask	資料來源	

決定要從哪個範例開始後，請點選上方的「Deploy on Cloud Run」(部署至 Cloud Run) 按鈕。系統會啟動 [Cloud Shell](#) (雲端虛擬機器的網頁版控制台)，並在其中複製來源、建構可部署的套件 (Docker 容器映像檔)，然後上傳至 [Google Container Registry](#)，最後部署到 [Cloud Run](#)。

系統詢問時，請指定 `us-central1` 區域。

下方的螢幕截圖顯示微服務建構和部署的 *Cloud Shell* 輸出內容

```

t [PROJECT_ID]"
demo@cloudshell:~$ cloudshell_open --repo_url "https://github.com/jamesward/hello-netcat.git" --page "editor"
[ ✓ ] Cloned git repository https://github.com/jamesward/hello-netcat.git.
[ ✓ ] Queried list of your GCP projects
[ ? ] Would you like to use existing GCP project jw-demo to deploy this app? Yes
[ ✓ ] Enabled Cloud Run API on project jw-demo.
[ ? ] Choose a region to deploy this application: us-central1
[ ! ] Attempting to build this application with its Dockerfile...
[ ! ] FYI, running the following command:
      docker build -t gcr.io/jw-demo/hello-netcat .
[ ✓ ] Built container image gcr.io/jw-demo/hello-netcat
[ ! ] FYI, running the following command:
      docker push gcr.io/jw-demo/hello-netcat
[ ✓ ] Pushed container image to Google Container Registry.
[ ! ] FYI, running the following command:
      gcloud run deploy hello-netcat\
        --project=jw-demo\
        --platform=managed\
        --region=us-central1\
        --image=gcr.io/jw-demo/hello-netcat\
        --memory=512Mi\
        --allow-unauthenticated
[ ✓ ] Successfully deployed service hello-netcat to Cloud Run.
* This application is billed only when it's handling requests.
* Manage this application at Cloud Console:
  https://console.cloud.google.com/run?project=jw-demo
* Learn more about Cloud Run:
  https://cloud.google.com/run/docs
[ ✓ ] Your application is now live here:
      https://hello-netcat-weurlhfjmq-uc.a.run.app
demo@cloudshell:~$ █

```

確認微服務是否正常運作

在 Cloud Shell 中，您可以向新部署的微服務提出要求，並將 `YOUR_SERVICE_URL` 替換為服務的網址 (位於 Cloud Shell 中「Your application is now live here」這一行之後)：

```
curl -d '{
  "_links": {
    "self": {
      "href": "https://foo.com"
    }
  },
  "arena": {
    "dims": [4,3],
    "state": {
      "https://foo.com": {
        "x": 0,
        "y": 0,
        "direction": "N",
        "wasHit": false,
        "score": 0
      }
    }
  }
}' -H "Content-Type: application/json" -X POST -w "\n" \
https://YOUR_SERVICE_URL
```

您應該會看到 **F**、**L**、**R** 或 **T** 的回應字串。

步驟 4 · 約 5 分鐘

4. 要求將應用程式納入競技場

如要加入 Rainbow Rumpus，必須先加入競技場。開啟 rainbowrumpus.dev，然後按一下競技場的「加入」，並提供微服務網址。

5. 進行及部署變更

如要進行變更，您必須在 Cloud Shell 中設定 GCP 專案和所用範例的相關資訊。首先列出您的 GCP 專案：

```
gcloud projects list
```

您可能只有一個專案。從第一欄複製 `PROJECT_ID`，然後貼到下列指令中 (將 `YOUR_PROJECT_ID` 替換為實際的專案 ID)，以便設定環境變數，供後續指令使用：

```
export PROJECT_ID=YOUR_PROJECT_ID
```

現在請為您使用的範例設定另一個環境變數，以便在後續指令中指定正確的目錄和服務名稱：

```
# Copy and paste ONLY ONE of these
export SAMPLE=kotlin-micronaut
export SAMPLE=kotlin-quarkus
export SAMPLE=kotlin-springboot
export SAMPLE=java-quarkus
export SAMPLE=java-springboot
export SAMPLE=go
export SAMPLE=nodejs
export SAMPLE=python
```

現在，您可以在 Cloud Shell 中編輯微服務的來源。如要開啟 Cloud Shell 網頁版編輯器，請執行下列指令：

```
cloudshell edit cloudbowl-microservice-game/samples/$SAMPLE/README.md
```

接著，系統會顯示變更的進一步操作說明。

The screenshot shows the Cloud Shell web interface. The top part is a code editor with a file explorer on the left showing 'devnexus-samples' and 'README-cloudshell.txt'. The main editor area shows a file named 'README.md' with the following content:

```
1 Cloud Bowl - Samples Java SpringBoot
2 -----
3
4 Run Locally:
5 ```
6 ./mvnw spring-boot:run
7 ```
8
9 [!Run on Google Cloud](https://deploy.cloud.run/button.svg)(https
10
```

The bottom part of the screenshot shows the terminal output:

```
--memory=512Mi\
--allow-unauthenticated
[ ✓ ] Successfully deployed service java-springboot to Cloud Run.
* This application is billed only when it's handling requests.
* Manage this application at Cloud Console:
  https://console.cloud.google.com/run?project=jw-demo
* Learn more about Cloud Run:
  https://cloud.google.com/run/docs
[ ✓ ] Your application is now live here:
  https://java-springboot-weurlhfjng-uc.a.run.app
demo@cloudshell:~$ gcloud projects list
PROJECT_ID  NAME  PROJECT_NUMBER
jw-demo     demo  247936028638
demo@cloudshell:~$ export PROJECT_ID=jw-demo
demo@cloudshell:~$ export SAMPLE=java-springboot
demo@cloudshell:~$ cloudshell edit samples/$SAMPLE/README.md
demo@cloudshell:~$
```

Cloud Shell 編輯器已開啟範例專案

儲存變更後，請使用 `README.md` 檔案中的指令在 Cloud Shell 中啟動應用程式，但請先確認您位於 Cloud Shell 中的正確範例目錄：

```
cd cloudbowl-microservice-game/samples/$SAMPLE
```

應用程式執行後，請開啟新的 Cloud Shell 分頁，並使用 `curl` 測試服務：

```
curl -d '{
  "_links": {
    "self": {
      "href": "https://foo.com"
    }
  },
  "arena": {
    "dims": [4,3],
    "state": {
      "https://foo.com": {
        "x": 0,
        "y": 0,
        "direction": "N",
        "wasHit": false,
        "score": 0
      }
    }
  }
}' -H "Content-Type: application/json" -X POST -w "\n" \
http://localhost:8080
```

準備好部署變更時，請使用 `pack` 指令在 Cloud Shell 中建構專案。這項指令會使用建構套件偵測專案類型、編譯專案，並建立可部署的構件 (Docker 容器映像檔)。

```
# Make sure you are in a Cloud Shell tab where you set the PROJECT_ID
# and SAMPLE env vars. Otherwise, set them again.
pack build gcr.io/$PROJECT_ID/$SAMPLE \
  --path ~/cloudbowl-microservice-game/samples/$SAMPLE \
  --builder gcr.io/buildpacks/builder
```

容器映像檔建立完成後，請使用 `docker` 指令 (在 Cloud Shell 中) 將容器映像檔推送至 Google Container Registry，以便 Cloud Run 存取：

```
docker push gcr.io/$PROJECT_ID/$SAMPLE
```

現在將新版本部署至 Cloud Run：

```
gcloud run deploy $SAMPLE \
  --project=$PROJECT_ID \
  --platform=managed \
  --region=us-central1 \
  --image=gcr.io/$PROJECT_ID/$SAMPLE \
  --allow-unauthenticated
```

現在競技場就會使用新版本！

6. 在本機開發 (選用)

您可以按照下列步驟，使用自己的 IDE 在本機處理專案：

1. [在 Cloud Shell 中] 將範例壓縮成 ZIP 檔案：

```
# Make sure the SAMPLE env var is still set. If not, re-set it.
cd ~/cloudbowl-microservice-game/samples
zip -r cloudbowl-sample.zip $SAMPLE
```

2. [在 Cloud Shell 中] 將 ZIP 檔案下載至電腦：

```
cloudshell download-file cloudbowl-sample.zip
```

3. [在電腦上] 解壓縮檔案，然後進行及測試變更

4. [在您的電腦上] [安裝 gcloud CLI](#)

5. [在您的電腦上] 登入 Google Cloud：

```
gcloud auth login
```

6. [在您的電腦上] 將環境變數 `PROJECT_ID` 和 `SAMPLE` 設為與 Cloud Shell 相同的值。

7. [在本機上] 使用 Cloud Build 建構容器 (從根專案目錄)：

```
gcloud alpha builds submit . \
  --pack=image=gcr.io/$PROJECT_ID/$SAMPLE \
  --project=$PROJECT_ID
```

8. [在您的電腦上] 部署新容器：

```
gcloud run deploy $SAMPLE \
  --project=$PROJECT_ID \
  --platform=managed \
  --region=us-central1 \
  --image=gcr.io/$PROJECT_ID/$SAMPLE \
  --allow-unauthenticated
```

7. 持續推送軟體更新

設定 SCM

設定 GitHub，與團隊協作處理微服務：

1. [登入 GitHub](#)
2. [建立新的存放區](#)
3. 如果您是使用本機電腦工作，可以透過 git 指令列介面 (CLI) 或 GitHub Desktop GUI 應用程式 (Windows 或 Mac) 執行作業。如果您使用 Cloud Shell，則必須使用 git CLI。如要在 GitHub 上取得微服務的程式碼，請按照 CLI 或 GitHub Desktop 的操作說明進行。

使用 git CLI 推送程式碼

1. 按照[這項說明](#)，透過 HTTPS 和個人存取權杖使用 Git
2. 選擇「repo」範圍
3. 設定 Git：

```
git config --global credential.helper \
  'cache --timeout=172800'
git config --global push.default current
git config --global user.email "YOUR@EMAIL"
git config --global user.name "YOUR NAME"
```

4. 設定 GitHub 機構和存放區的环境變數 (`https://github.com/ORG/REPO`)

```
export GITHUB_ORG=YOUR_GITHUB_ORG
export GITHUB_REPO=YOUR_GITHUB_REPO
```

5. 將程式碼推送到新存放區

```
# Make sure the SAMPLE env var is still set. If not, re-set it.
cd ~/cloudbowl-microservice-game/samples/$SAMPLE
git init
git add .
git commit -m init
git remote add origin https://github.com/$GITHUB_ORG/$GITHUB_REPO.git
git branch -M main

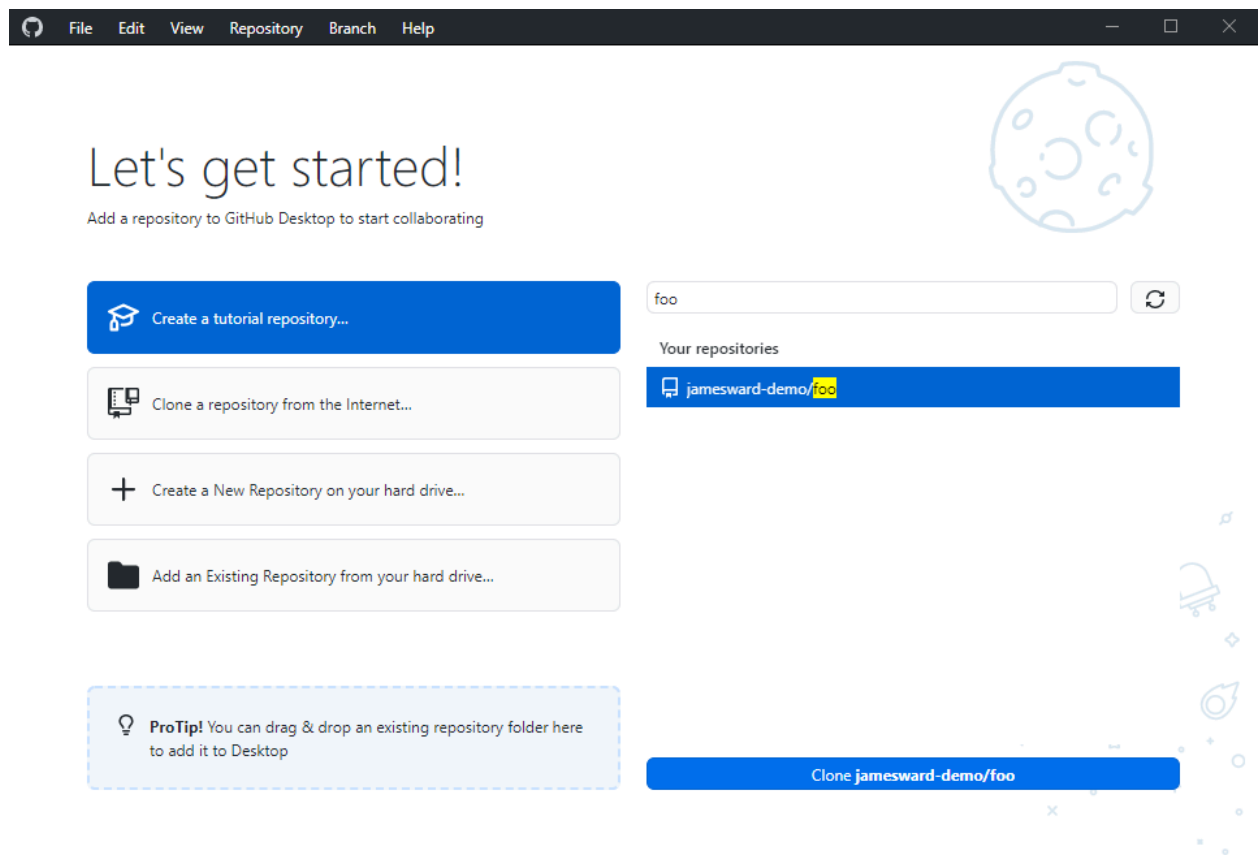
# This will now ask for your GitHub username & password
# for the password use the personal access token
git push -u origin main
```

6. 進行變更後，您可以將變更提交並推送至 GitHub：

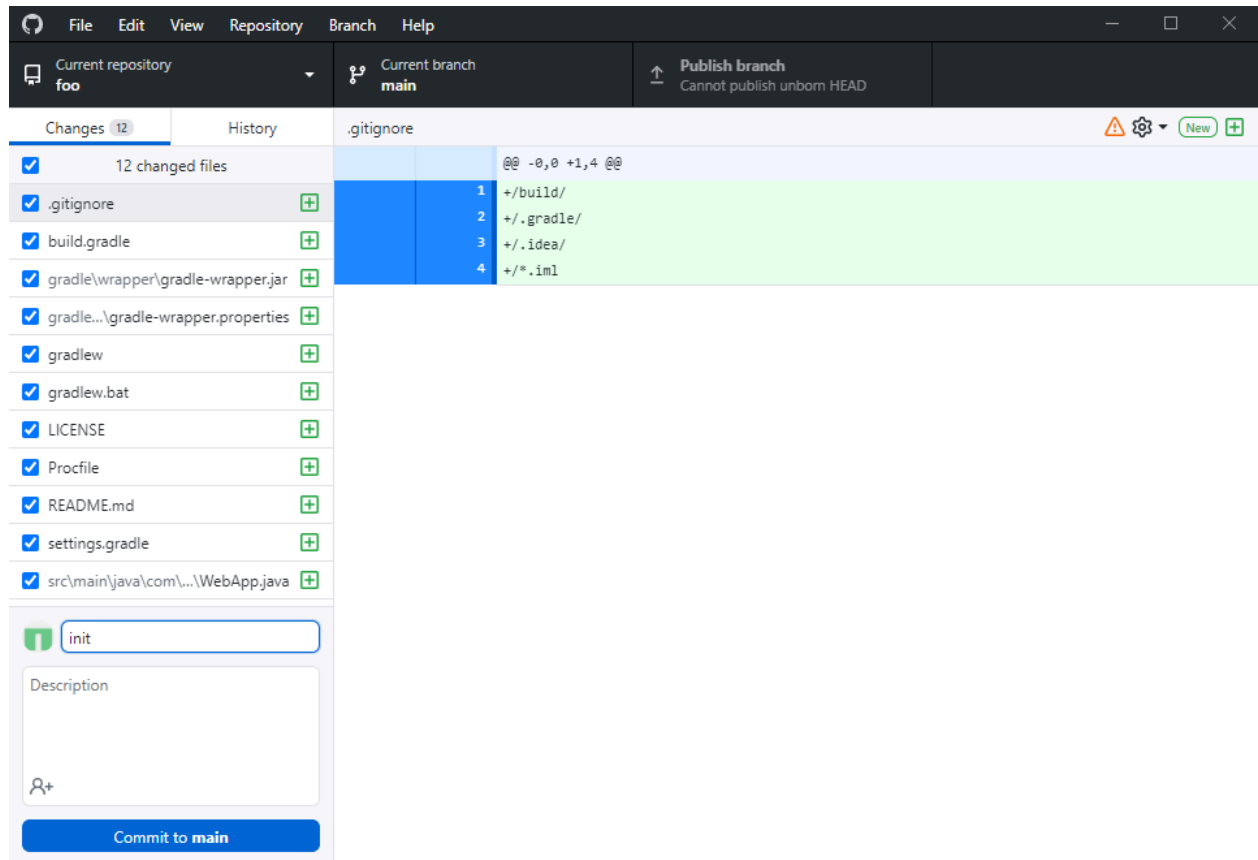
```
git add .  
git status  
git diff --staged  
git commit -am "my changes"  
git push
```

使用 GitHub Desktop 推送程式碼

1. 按照先前「在本機開發」實驗室的操作說明下載程式碼
2. 安裝 [GitHub Desktop](#)、啟動並登入
3. 複製新建的存放區



4. 開啟檔案總管，然後將專案複製到新存放區
5. 修訂變更

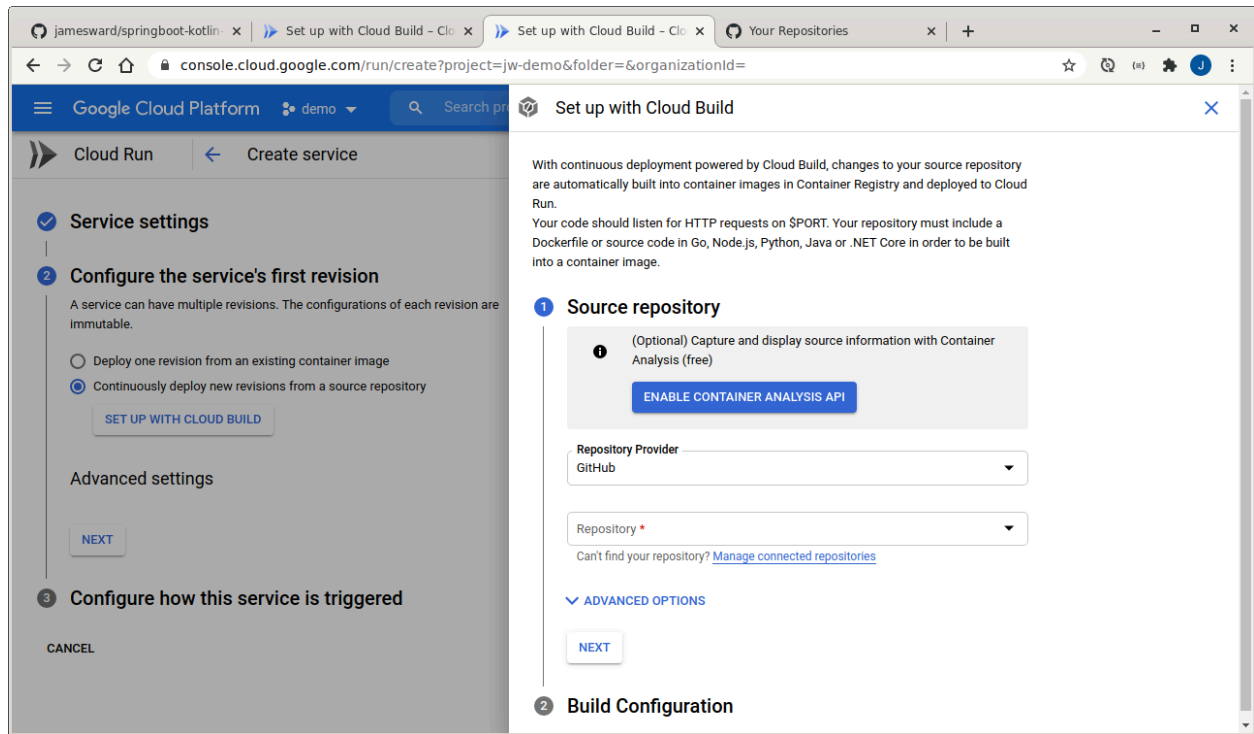


6. 將主要分支版本發布至 GitHub

設定 Cloud Run 持續部署

在 GitHub 上設定 SCM 後，您現在可以設定持續交付，這樣每當有新的提交內容推送至 `main` 分支版本時，Cloud Build 就會自動建構及部署變更。您也可以新增持續整合，在部署前執行測試，但由於現成範例不含任何測試，因此這個步驟已留給您練習。

1. 在 [Cloud 控制台](#) 中，前往 Cloud Run 服務
2. 按一下「設定持續部署」按鈕
3. 透過 GitHub 驗證，並選取微服務的存放區



4. 選取 GitHub 存放區，並將分支版本設為：`^main$`

✓

Source repository

2

Build Configuration

Branch *

`^main$`

Use a regular expression to match to a specific branch [Learn more](#)

Matches the branch: main

Build Type

☐ Dockerfile

☒ Go, Node.js, Python, Java or .NET Core via [Google Cloud Buildpacks](#) [↗](#)

Build context directory

/

The directory will be used as the Buildpack build context.

Entrypoint

Command to start the server, leave blank to use [default behavior](#) [?](#)

SAVE

5. 將建構類型設為使用 Buildpacks

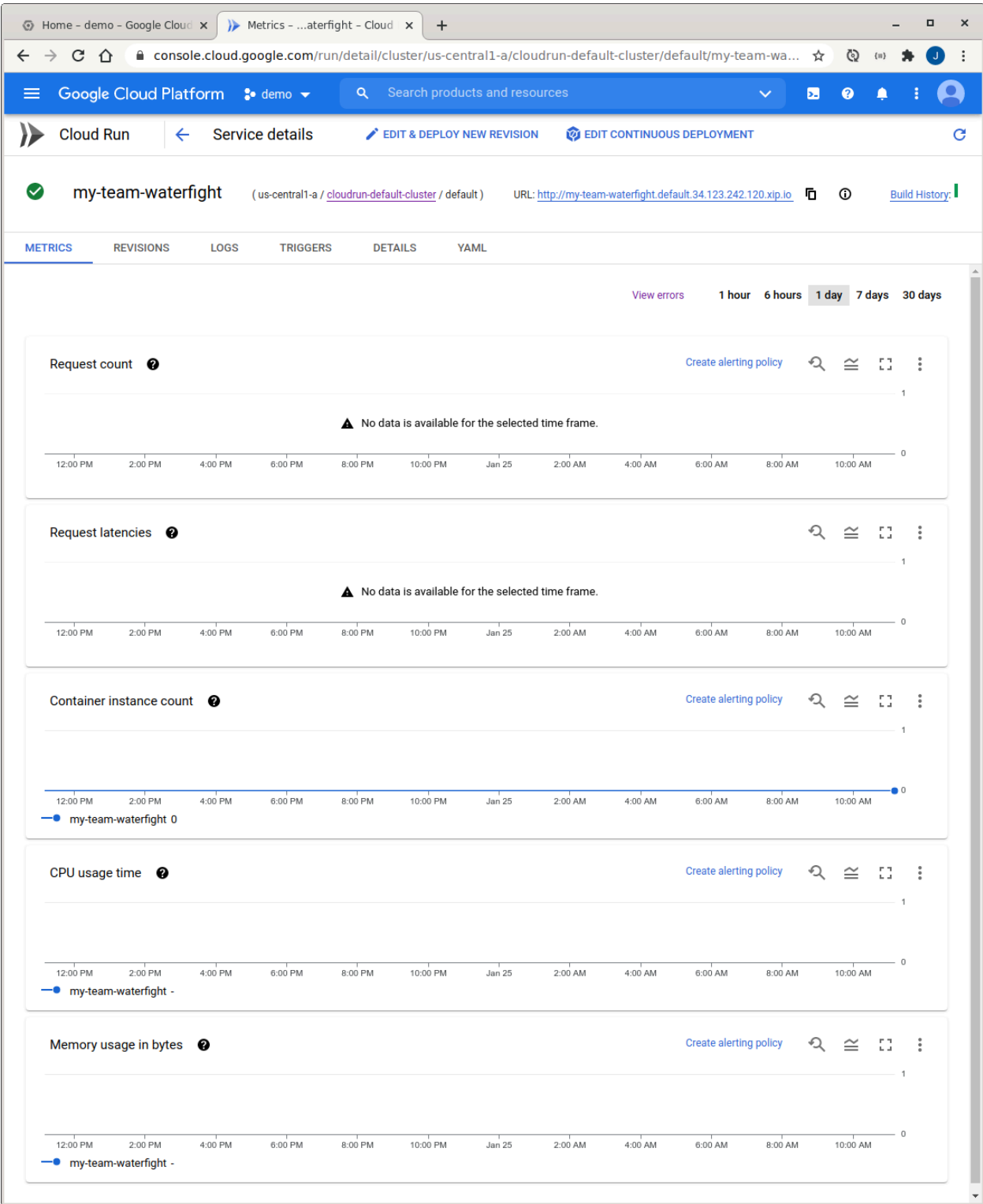
6. 按一下「儲存」設定持續部署。

8. 觀測能力

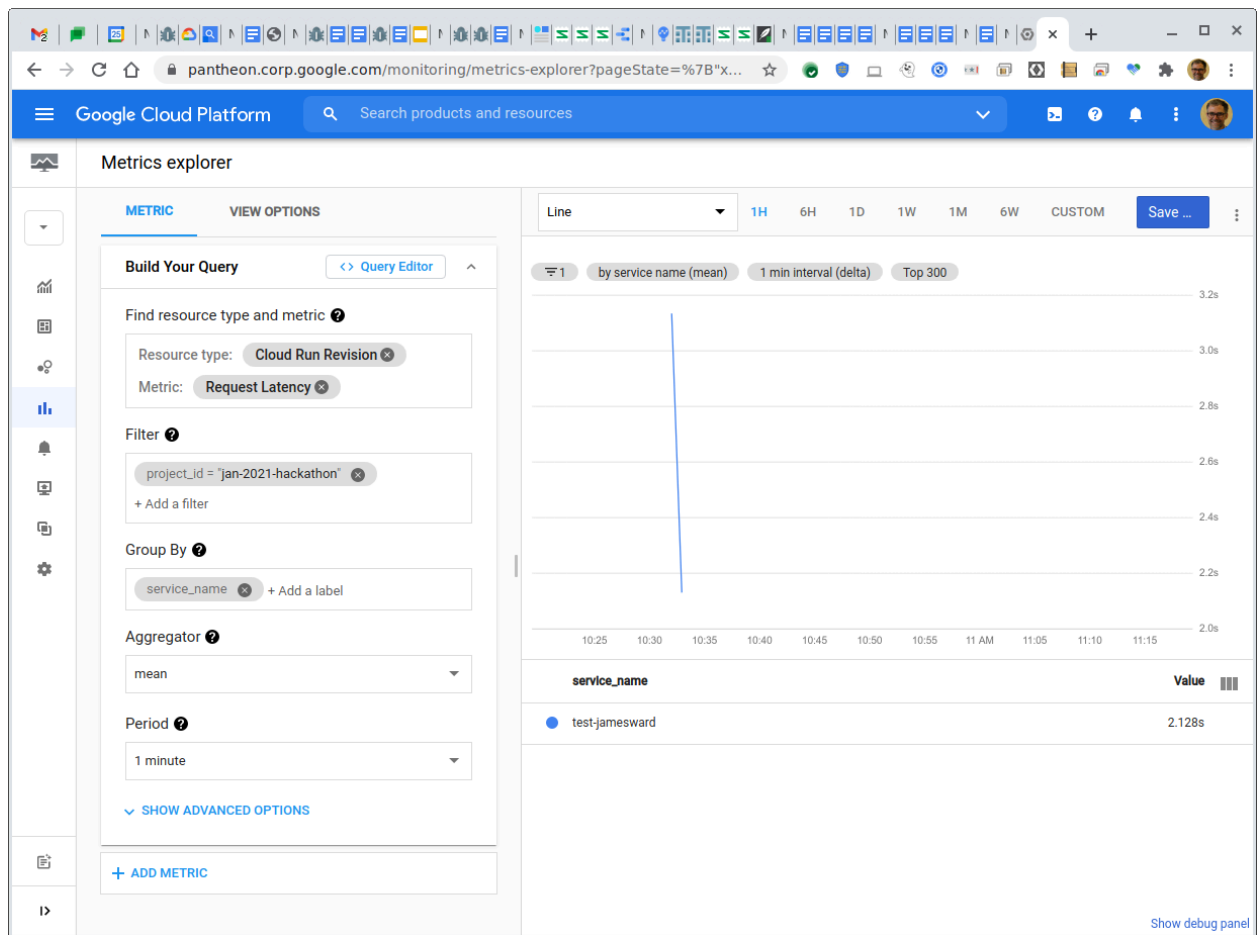
東西會壞掉。可觀測性讓我們能夠掌握何時發生這種情況，並診斷原因。指標會顯示服務的健康狀態和使用情形相關資料。記錄會顯示服務發出的手動檢測資訊。快訊可讓我們在發生問題時收到通知。以下將進一步說明各項功能。

指標

- 1. 在 [Cloud Run 服務清單](#) 中找到您的服務
- 2. 按一下服務名稱，即可前往指標資訊主頁

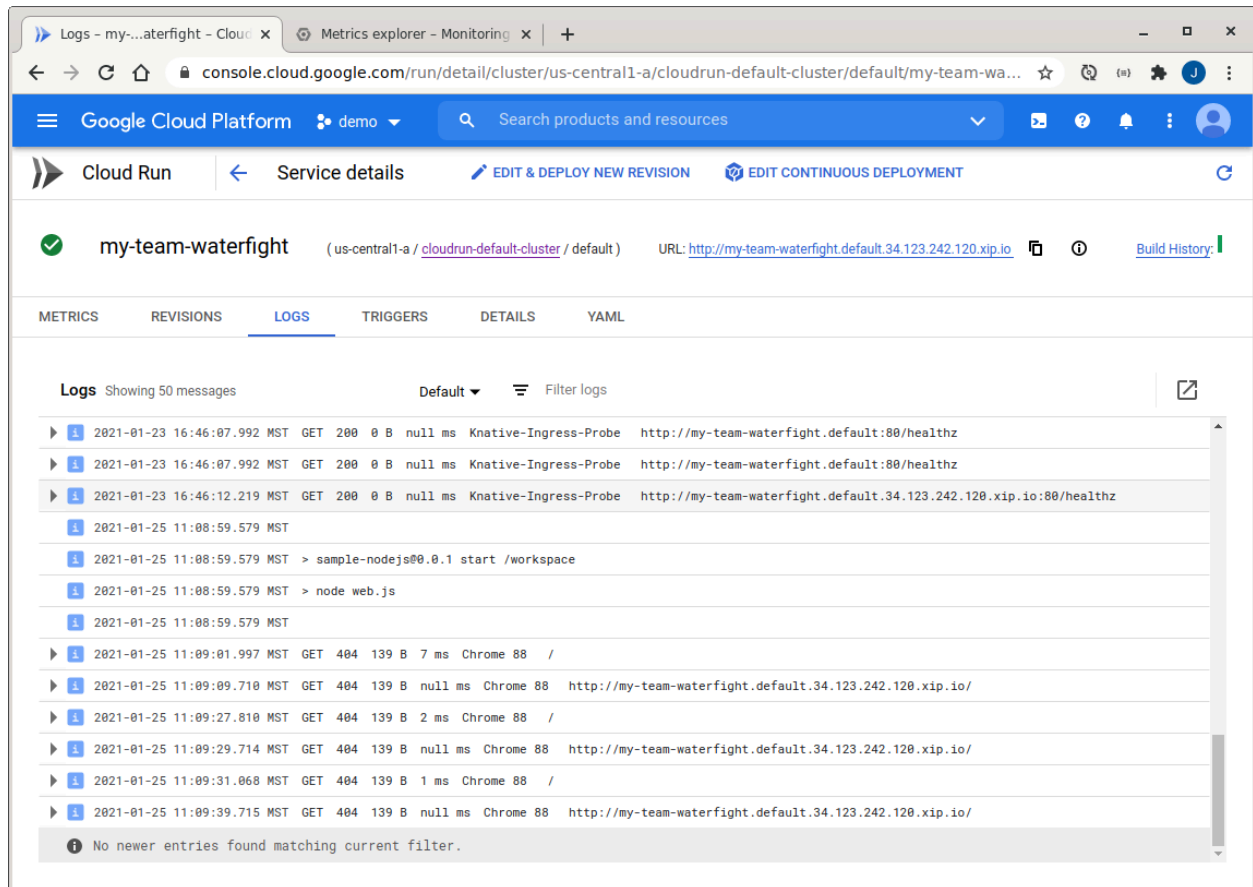


3. 按一下指標的：選單，然後選取「在 Metrics Explorer 中查看」
4. 現在可以變更資源指標、篩選器、分組和其他選項。舉例來說，您可以[查看所有服務的平均服務延遲時間](#)：



記錄

服務的 STDOUT 輸出內容會傳送至 Google Cloud Logging 系統。您可以從 Cloud Run 服務管理頁面存取基本記錄檢視畫面，例如：



在 Cloud Run 記錄中，您可以依嚴重性篩選記錄。如要享有更多彈性，請按一下：



快訊

1. 為服務建立健康狀態檢查網址。
2. 如果是 Spring Boot，只要新增下列依附元件即可：

```
org.springframework.boot:spring-boot-starter-actuator
```

2. 建立或更新 `src/main/resources/application.properties`，並停用磁碟空間檢查：

```
management.health.diskspace.enabled=false
```

2. 建立運作時間快訊，並指定通訊協定、主機名稱和路徑。如果是 Spring Boot，路徑為：`/actuator/health`
3. 測試警告

Create Uptime Check - Monit xMetrics - ...aterfight - Cloud x

console.cloud.google.com/monitoring/uptime?project=jw-demo

Google Cloud PlatformSearch products and resources

Uptime checksCREATE UPTIME CHECK

Filter table

Display Name ↑

No rows to display

1>

Create Uptime Check

1Title

Enter a name for the uptime check.

TitleMy Waterfight

2Target

Select the resource to be monitored.

ProtocolHTTPS

Resource Type

- ☒ URL ?
- ☐ App Engine ?
- ☐ Instance ?
- ☐ Elastic Load Balancer ?

Hostname*
gcp-credits-api-jmnwxz7saq-uc.a.run.app ?

Path
/actuator/health ?

URL ?
https://gcp-credits-api-jmnwxz7saq-uc.a.run.app/actuator/health

Check Frequency
1 minute ?

MORE TARGET OPTIONS

NEXT

3Response Validation

Specify data and how that data is to be compared to the actual response data.

4Alert & Notification

Define Uptime Check Alert Condition.

Responded with "200 (OK)" in 186 ms.

CREATETESTCANCEL

4. 建立快訊

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜！您已成功建構及部署微服務，可與其他微服務對戰！祝您好運！

參考文件

- [Cloud Run 說明文件](#)
-